

Full length article

FLSTM-ESD: A hybrid approach to fuzziness based long short-term memory network for email spam detection

Ekramul Haque Tusher^a, Mohd Arfian Ismail^{a,b} ^{*}

^a Faculty of Computing, University Malaysia Pahang Al-Sultan Abdullah, Pekan, 26600, Pahang, Malaysia

^b Center of Excellence for Artificial Intelligence & Data Science, University Malaysia Pahang Al-Sultan Abdullah, Lebuhraya Tun Razak, Gambang, 26600, Pahang, Malaysia

ARTICLE INFO

Keywords:

Fuzziness
Long short-term memory
Email spam
Detection
Deep learning

ABSTRACT

Email spam detection is essential for cybersecurity, as spam frequently contains phishing attempts, malware, and fraudulent content. Traditional rule-based systems and machine learning methods struggle to adapt to evolving spam tactics, resulting in declining detection accuracy. Deep learning (DL) methods have been explored due to their ability to model complex data patterns effectively. However, fuzzy systems, while interpretable and adaptable, often fail to classify complex datasets accurately. Long Short-Term Memory (LSTM) networks, known for sequential data processing, face challenges with memorizing noise, limiting their effectiveness in spam detection. This study introduces a hybrid approach, Fuzziness-based LSTM for Email Spam Detection (FLSTM-ESD), which integrates fuzzy logic with LSTM networks. A pilot study compared four DL methods: Artificial Neural Network (ANN), LSTM, Gated Recurrent Unit (GRU), and Bidirectional LSTM (Bi-LSTM). LSTM emerged as the most effective for detecting complex spam patterns. To enhance performance, a fuzziness-based divide-and-conquer strategy was implemented, incorporating low and high fuzziness samples into the training process. Experimental results confirmed that FLSTM-ESD outperformed other DL methods, achieving 94.9% accuracy on the Spambase dataset, 96.84% on Spamassassin, and 97.09% on Lingspam. FLSTM-ESD establishes itself as a state-of-the-art method for spam detection, surpassing existing approaches by integrating fuzzy logic with LSTM to improve accuracy and handle evolving spam patterns. These results show FLSTM-ESD's robustness for real-world email spam detection applications.

1. Introduction

Email has become widely prevalent as a fundamental mode of communication, including both personal and professional spheres, and has firmly established itself as a vital resource in our everyday existence [1]. It enables swift and effective communication across vast distances and time zones, making it indispensable for various applications, including business, education, and personal correspondence [2]. However, this reliance on email has also led to an alarming rise in spam emails, posing significant challenges to the security and efficiency of email systems. Spam emails, often unsolicited, can carry harmful content such as phishing links, malware, or scams aimed at deceiving recipients. Not only do these spam emails consume valuable system resources like bandwidth and storage, but they also pose a direct threat to the security and privacy of users [3]. Consequently, the need for reliable and efficient email spam detection systems has grown increasingly critical, particularly as spammers continue to evolve their tactics. In the early days of spam detection, rule based filtering

techniques were the dominant approach [4]. These systems relied on predefined sets of rules to classify emails as either spam or ham based on certain characteristics, such as specific keywords or the presence of suspicious attachments [5–7]. While these rule-based systems were initially effective at blocking spam, they quickly became inadequate as spammers adapted their techniques to evade detection. The dynamic nature of spam, with spammers continuously adjusting their content to bypass filters, rendered static, rule-based methods ineffective [8]. The sophistication of modern spam emails, which often mimic legitimate communication or contain subtle variations to avoid detection, necessitates more advanced and adaptive solutions [9]. As a result, Machine Learning (ML) based approaches have emerged as a promising alternative.

Deep Learning (DL) has become vital for spam detection, leveraging patterns in large datasets to classify emails accurately [10]. DL methods, especially Long Short-Term Memory (LSTM) networks, excel in sequential data processing tasks like email classification [11–14].

^{*} Corresponding author.

E-mail address: arfian@ump.edu.my (M.A. Ismail).

<https://doi.org/10.1016/j.aei.2026.104597>

Received 22 August 2025; Received in revised form 29 January 2026; Accepted 10 March 2026

Available online 13 March 2026

1474-0346/© 2026 Published by Elsevier Ltd.

LSTM network is particularly well-suited for this task because it can capture contextual relationships between words and phrases in an email while preserving long-range dependencies across the sequence. This capability is crucial for accurately determining whether an email is spam or legitimate, especially when subtle contextual clues play a significant role in the classification [15]. However, LSTM struggles with ambiguous cases where subtle nuances blur the line between spam and legitimate emails [16]. This limitation stems from the binary nature of traditional classifiers, leading to potential misclassifications, such as false positives or false negatives [17]. Addressing these challenges requires methods that account for uncertainty and vagueness in real-world scenarios [18]. Fuzzy set theory, introduced by Lotfi A. Zadeh in 1965, addresses the vagueness and uncertainty inherent in many real-world scenarios by allowing partial membership in categories [19]. In spam detection, emails can partially belong to both spam and ham categories, making fuzzy logic ideal for handling ambiguous data and enabling nuanced decision-making [20]. While fuzzy sets improve classification by incorporating degrees of membership, determining fuzzy parameters manually is challenging in complex situations, necessitating automated approaches for better accuracy [21]. This flexibility makes fuzzy logic a robust tool for domains like email spam detection.

To enhance the adaptability and precision of fuzzy systems in spam detection, a fuzzy automated approach is essential. This ensures that adjustments to fuzzy modeling parameters are made automatically, reducing manual tuning efforts and improving system responsiveness to evolving spam tactics. One such advanced technique is the divide and conquer fuzzy membership vector optimization strategy. This method provides a systematic framework to tune the fuzzy system for superior results by decomposing the optimization problem into smaller, manageable subproblems that are solved independently and then combined. The fuzzy membership vector derived from this approach complements LSTM training, resulting in a highly accurate detection method with strong generalization capabilities. By integrating the fuzzy membership vector with the LSTM network, the method can learn more powerful features from emails. The membership vector acts as a guiding mechanism, helping the LSTM focus on meaningful patterns within the email data during training. This synergy combines the interpretative strength of fuzziness with the sequential learning capabilities of LSTM, offering a comprehensive solution to the challenges faced by traditional LSTM methods. The fuzziness introduces an interpretative layer to the method, enabling softer decision boundaries and reducing misclassification risks, especially in borderline cases. Meanwhile, LSTM's proficiency in capturing contextual dependencies ensures deeper semantic understanding of email content. The hybrid Fuzziness-based LSTM for Email Spam Detection (FLSTM-ESD) thus leverages this divide-and-conquer fuzzy membership optimization alongside LSTM's sequential learning to maintain robustness and adaptability in dynamic, real world email environments. By continuously learning from new data and refining fuzzy membership calculations, FLSTM-ESD is well-equipped to counter sophisticated evasion techniques and sustain high classification accuracy over time.

2. Data collection and dataset description

In the data collection for the research, datasets utilized in the experiment's execution were gathered. Reviewing popular applications in previous research helped in selecting high-quality, pertinent datasets that aligned well with the proposed method. Every selected dataset has two-class attributes related to spam emails. Spambase, Spamassassin, and Lingspam were the three benchmark spam datasets gathered to assess the proposed approach. The public databases from Kaggle, including Spamassassin and Lingspam, as well as the UCI Machine Learning Repository, which includes Spambase, were utilized. These databases are accessible online at <https://archive.ics.uci.edu> and <https://www.kaggle.com>. Table 1 briefly summarizes the collected email spam datasets. This data collection process ensured the acquisition

Table 1

Explanation of all collected email spam datasets.

Datasets	Instances (Rows)	Number of attributes	Number of classes
Spambase	4601	58	2
SpamAssassin	4198	4	2
Ling Spam	2893	3	2

of appropriate standard datasets frequently utilized in similar studies, facilitating effective performance analysis of the proposed methodology for email spam detection.

The first dataset, Spambase, contained two classes: spam and ham. It had 55 continuous, real-valued features, two continuous, integer-valued features, and one nominal feature. The dataset comprised 4601 instances, with 2788 labeled as ham and 1813 as spam. Instances and features were represented using 58-dimensional vectors. Of the 57 attributes, 48 were words extracted from the original messages without any stop word removal or stemming, considered the most discriminative terms for the spam class. The remaining six attributes captured the percentage of occurrence of special characters such as ';', '(', '[', '!', '\$', and '#'. Finally, the class label indicated whether an instance was spam, represented by one, or ham, represented by zero.

The second dataset used in this study is Spamassassin, which contains two classes: spam and ham. This dataset consists of 4198 total instances, with 2800 labeled as ham and 1398 labeled as spam. The Spamassassin dataset is a public collection of email spam corpora made available for researchers. It contains four attributes: label, label_num, text, and number. The label and label_num attributes indicate whether each instance is spam or ham. The text attribute provides the email content, while the number contains integers. The Spamassassin corpus provides a source of real email data with spam or ham labels.

Finally, the third dataset used is Lingspam, a collection of 2893 spam and non-spam messages curated from the Linguist List mailing list on linguistics. This list covers scientific topics like job postings, research, and software. The Lingspam dataset contains legitimate and spam emails in English, with 2412 labeled as ham and 481 as spam. It has three attributes: subject, message, and label. The subject and message provide the raw email content, while the label indicates if each instance is spam (1) or ham (0).

2.1. Experimental environment

Software and hardware resources were essential to perform the documentation and complete the research effectively. The hardware requirement for this study included a high-performance laptop equipped with a Ryzen 5 processor, operating at a clock speed ranging from 2.1 GHz to 3.7 GHz. Additionally, the laptop featured a Radeon RX Vega 8 integrated GPU, which had 8 Compute Units (512 Shaders) and a maximum clock speed of 1200 MHz, along with 12 GB of Random Access Memory (RAM). This hardware setup was chosen to ensure efficient processing power for running data-intensive tasks and managing multitasking needs. The laptop served multiple functions, from conducting thorough documentation work and accessing online resources to supporting software necessary for research activities. It facilitated all aspects of documentation, including data collection, literature review, and synthesis of information accessed from the Internet. For the research's technical requirements, particularly in implementing and testing code, Jupyter software was used to run Python. This software provided an interactive environment suitable for coding, debugging, and analyzing results, making it an essential tool for managing and executing the research methodology effectively.

Table 2
Optimal parameter values for different datasets and models.

Dataset	Parameter	Utilized values	Optimal value for ANN	Optimal value for LSTM	Optimal value for GRU	Optimal value for Bi-LSTM
Spambase	Batch size	8, 16, 32	8	16	16	16
	Dropout	0.1, 0.2, 0.3, 0.4	0.2	0.2	0.2	0.2
	Number of epochs	30, 40, 50, 60	30	50	40	40
	Optimizer	Adam, RMSprop	Adam	RMSprop	RMSprop	RMSprop
Lingspam and spamassassin	Batch size	8, 16, 32	16	16	16	16
	Dropout	0.1, 0.2, 0.3, 0.4	0.2	0.2	0.2	0.2
	Number of epochs	5, 10, 15, 20	10	15	10	20
	Optimizer	Adam, RMSprop	Adam	Adam	Adam	Adam

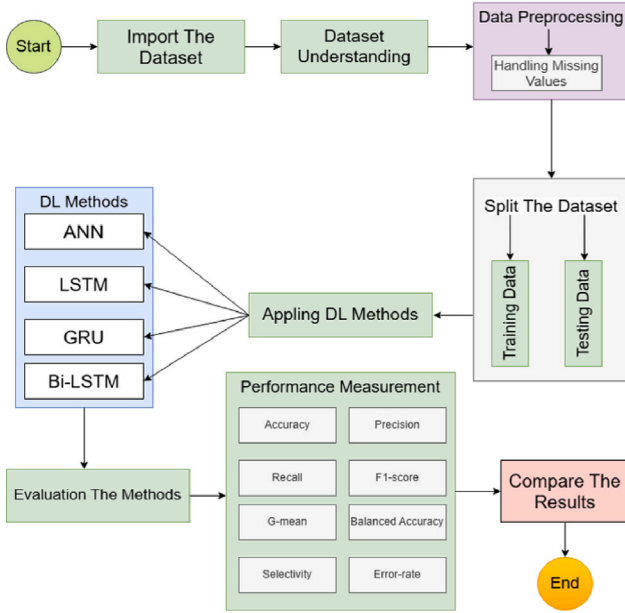


Fig. 1. Flowchart of the pilot study.

3. Methodology

3.1. Pilot study

The research started with a review of the current methods for detecting email spam, in order to study previous methods and identify limitations. Through careful analysis, ways were found to fix the problems with the present methods. The proposed methods were chosen by looking at how well they fit with the goals, objectives, and scope of the study. During the data collection phase email spam datasets were gathered to be used during experiment implementation. Subsequently, the datasets were used to compare how well different DL methods worked. Thereafter, the experiment implementation phase gave a detailed design of the proposed method and an evaluation of it. The pilot study for email spam detection, comparing four different methods to determine the optimal approach. The methods examined, based on a review of academic literature, were ANN, LSTM, GRU, and Bi-LSTM. The pilot study followed the process outlined in algorithm 1 and Fig. 1.

3.2. Hyperparameter optimization

Hyperparameter tuning plays a crucial role in optimizing DL methods for email spam detection. The selection of batch size, dropout rate, number of epochs, and optimizer significantly influences method performance. In this study, experiments were conducted using different hyper-parameters to determine the optimal values for ANN, LSTM, GRU, and Bi-LSTM across multiple datasets presented in Table 2.

Algorithm 1 Deep Learning Methods Comparison for Email Prediction

```

1: Import the dataset
2: dataset ← importDataset()
3: Understand the dataset
4: understandDataset(dataset)
5: Preprocess the data
6: handleMissingValues(dataset)
7: Split the dataset into training and testing data
8: trainingData, testingData ← splitDataset(dataset)
9: Apply deep learning methods
10: DL methods ← [ANN(), LSTM(), GRU(), Bi-LSTM()]
11: for each method in methods do
12:   trainMethod(method, trainingData)
13:   results ← evaluateMethod(method, testingData)
14:   Print performance metrics for each method
15:   print("Accuracy:", results.accuracy)
16:   print("Precision:", results.precision)
17:   print("Recall:", results.recall)
18:   print("F1-score:", results.f1Score)
19:   print("G-mean:", results.gMean)
20:   print("Balanced Accuracy:", results.balancedAccuracy)
21: end for
22: Compare the results of different methods and predict email (as per flowchart)
23: bestMethod ← compareResults(methods)
24: predictedEmail ← predictEmail(bestMethod, testingData)
  
```

3.3. Comparative analysis

The pilot study analyzed and experimented with four key DL methods. These methods were selected based on their prevalent use in prior academic literature for spam detection. The comparative evaluation focused on applying each technique on three public email datasets - spambase, spamassassin, and lingspam. The performance of each method on the datasets was measured using various metrics like accuracy, precision, recall, f1-score, g-mean, balanced accuracy, selectivity, and error-rate. Spider plots visually compared the metrics across methods on each dataset to identify the optimal method.

The results on the spambase dataset showed a different outcome. As seen in Fig. 2, the accuracy values of all four DL methods were very similar and achieved high accuracy overall. The radar plot compares the performance metrics of the ANN, LSTM, GRU and Bi-LSTM methods trained and evaluated on the spambase dataset. Eight different metrics were used to quantify the methods' spam detection capabilities from various perspectives. These include accuracy, precision, recall, f1-score, g-mean, balanced accuracy, selectivity, and error rate.

The results on the spambase dataset indicate that the LSTM method achieved the highest performance across most evaluation metrics compared to the other methods. As summarized in Table 3, LSTM obtained

Table 3
Comparison results of all methods for spambase dataset.

Methods	Accuracy	Precision	Recall	F1-score	G-mean	Balanced accuracy	Selectivity	Error-rate
ANN	93.6	94.7	89.45	92.00	92.91	92.70	96.50	6.40
LSTM	94.03	90.46	92.86	91.65	93.04	93.02	93.19	5.97
GRU	91.64	94.53	89.62	92.00	93.04	93.09	96.58	8.36
Bi-LSTM	92.4	92.47	87.97	90.16	91.72	91.80	95.63	7.60

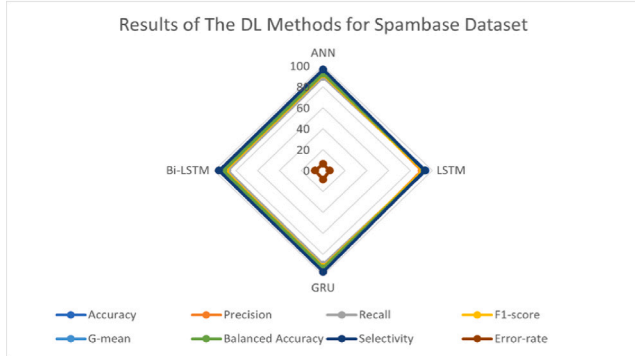


Fig. 2. Results of The DL methods for spambase dataset.

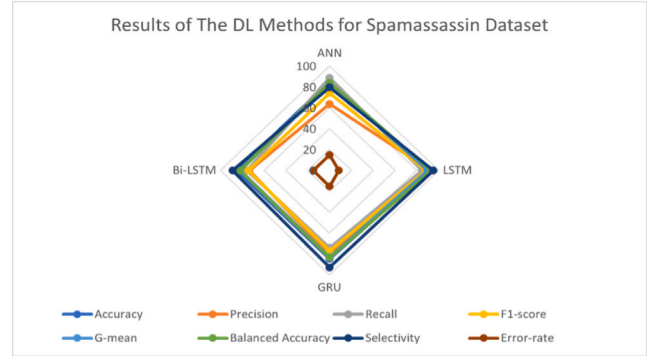


Fig. 3. Results of the DL methods for spamassassin dataset.

Table 4
The summary results obtained by DL methods on the spambase dataset.

Methods	Best accuracy	Average accuracy	Worst accuracy	Computational times (s)	SD
ANN	93.6	92.94	91.92	728	0.58
LSTM	94.03	93.51	93.19	652	0.32
GRU	91.64	90.99	89.84	683	0.67
Bi-LSTM	92.4	91.81	91.29	749	0.45

the highest accuracy of 94.03%, recall of 92.86%, g-mean of 93.04% and the lowest error rate of 5.97%. Out of the eight metrics used to assess the methods, LSTM outperformed the others in four—accuracy, recall, g-mean, and error rate. The LSTM architecture's ability to retain long-term context information appears better suited for the spambase dataset. The other methods—ANN, GRU, and Bi-LSTM also performed reasonably well, but LSTM emerged as the top performer based on the majority of metrics.

A comparison of DL methods (ANN, LSTM, GRU, and Bi-LSTM) on the spambase dataset, highlighting accuracy and computational time presented in Table 4. LSTM achieved the highest best accuracy of 94.03% and had the lowest SD of 0.32, indicating consistent performance across runs. It also required the least computational time at 652 s, making it both accurate and efficient. While ANN and Bi-LSTM performed adequately with best accuracies of 93.6% and 92.4%, respectively, they demanded more computational resources and exhibited slightly higher variability. GRU had the lowest worst accuracy 89.84% and the highest SD 0.67, showing more fluctuation in its performance.

On the spamassassin dataset, Fig. 3 shows the LSTM method achieved the highest performance across all evaluation metrics compared to the other models. The LSTM outperformed ANN, GRU, and Bi-LSTM in terms of metrics like accuracy, precision, recall, F1-score, and others. ANN, GRU and Bi-LSTM performed similarly, with minor differences in results. Their performance was reasonable, but LSTM consistently emerged as the top performer on spamassassin data.

The comparison results summary of the DL methods is shown in Table 5. The LSTM method obtained the highest accuracy of 91.53% compared to the other methods. Specifically, LSTM achieved an accuracy of 91.53%, precision of 92.60%, f1-score of 88.55%, g-mean of 89.86%, balanced accuracy of 93.02%, selectivity of 95.16%, and error-rate of 8.47%. Out of all the models, LSTM consistently achieved the top performance across the various evaluation metrics.

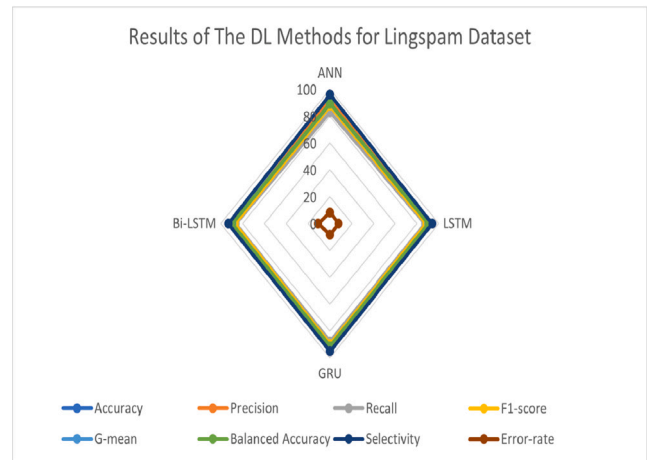


Fig. 4. Results of the DL methods for lingspam dataset.

The performance of DL methods (ANN, LSTM, GRU, and Bi-LSTM) on the spamassassin dataset, focusing on accuracy and computational time summarized in Table 6. LSTM achieved the highest best accuracy 91.53% and the lowest SD 0.18, indicating consistent and reliable performance. Although it required 1207 s, which was longer than some other methods, its accuracy and stability made it the most effective model overall. ANN and Bi-LSTM showed decent performance, with best accuracies of 85.21% and 85.12%, respectively, but required significantly longer computational times, especially Bi-LSTM at 1347 s. GRU had the lowest accuracy performance, with the best accuracy of 84.73% and the highest variability SD of 0.53, suggesting less reliability.

On the lingspam dataset, Fig. 4 shows that the accuracy of all the DL methods was comparable and achieved over 90%, except for Bi-LSTM. Specifically, ANN, LSTM, and GRU obtained similar accuracy scores above 90%, while Bi-LSTM was slightly lower at 89%. Though the methods performed similarly, LSTM again emerged with the top accuracy result.

Table 7 summarizes the results of the DL methods on the lingspam dataset. The LSTM method achieved the highest accuracy of 92.07% compared to the other methods. Specifically, the accuracy scores were

Table 5

Comparison results of all methods for spamassassin dataset.

Methods	Accuracy	Precision	Recall	F1-score	G-mean	Balanced accuracy	Selectivity	Error-rate
ANN	85.21	63.53	88.55	73.98	83.93	84.04	79.54	14.79
LSTM	91.53	92.60	84.85	88.55	89.86	93.02	95.16	8.47
GRU	84.73	80.58	74.04	77.19	82.89	83.45	92.82	15.27
Bi-LSTM	85.12	73.58	75.76	74.63	82.13	82.40	89.03	14.88

Table 6

The summary results obtained by DL methods on the spamassassin dataset.

Methods	Best accuracy	Average accuracy	Worst accuracy	Computational times (s)	SD
ANN	85.21	84.62	83.94	1172	0.51
LSTM	91.53	91.28	90.96	1207	0.18
GRU	84.73	84.06	83.29	1196	0.53
Bi-LSTM	85.12	84.87	84.20	1347	0.35

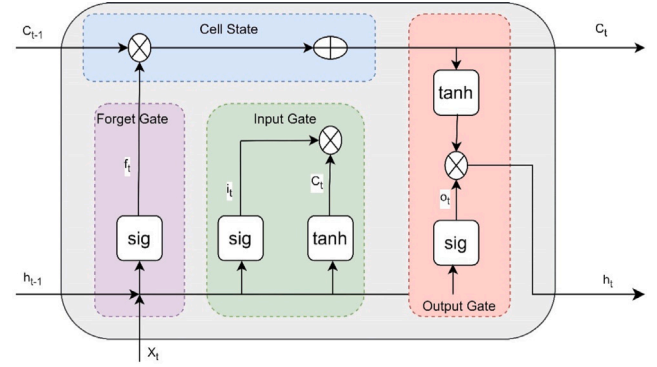
92.07% for LSTM, 91.64% for ANN, 91.86% for GRU, and 89.04% for Bi-LSTM. While GRU had the best performance on metrics like recall, f1-score, g-mean and balanced accuracy, LSTM obtained the top accuracy and lowest error rate.

Table 8 provides an overview of DL methods (ANN, LSTM, GRU, and Bi-LSTM) on the lingspam dataset, highlighting accuracy and computational duration. LSTM achieved the highest best accuracy at 92.07% and maintained the lowest SD 0.21, showing consistent performance. It also required the least computational time at 1263 s, making it both efficient and reliable. ANN followed with the best accuracy of 91.64% and an average computational time of 1303 s, while GRU exhibited the highest SD 0.69, indicating more variability and a slightly lower average accuracy of 91.17%. Bi-LSTM had the lowest accuracy and the longest computational time 1447 s, making it the least efficient among the methods.

Additionally, 30 independent runs were conducted as part of the statistical analysis to assess whether the results produced by all methods differed significantly from each other [22]. The purpose of doing a t-test on accuracy values from 30 runs of DL methods is to ascertain whether the performance disparity between the two methods is statistically significant. Accuracy values are obtained from 30 independent trials. This iterative assessment yields a more dependable estimation of the method's mean performance. As discussed in the previous chapter, a paired t-test was employed to compare the outcomes of two distinct methods in the pilot study. Tables 9, 10, and 11 present the fitness values obtained from both statistical tests for the Spambase, SpamAssassin, and LingSpam datasets, respectively. The significance level was set to $\alpha = 0.05$. Thus, if the p -value for a pairwise comparison between two methods was greater than 0.05, the difference was considered not statistically significant; otherwise, it was deemed statistically significant.

According to the statistical tests, the null hypothesis (H_0) stated that there is no statistically significant difference between the two specified DL methods being compared, whereas the alternative hypothesis (H_1) stated that a statistically significant difference exists between their performances. The results show that, for all datasets (Spambase, SpamAssassin, and LingSpam), the p -values obtained from the paired t -tests were lower than the significance level of $\alpha = 0.05$. Therefore, H_0 was rejected and H_1 was accepted. This indicates that the compared methods produced significantly different results, reflecting distinct performance characteristics. Hence, it can be concluded that the evaluated methods are statistically different from one another, highlighting variations in their effectiveness across the considered datasets.

The comparative evaluation conducted in this study analyzed four DL methods ANN, LSTM, GRU, and Bi-LSTM by applying them to three different spam email datasets. The methods were evaluated and compared based on performance metrics such as accuracy, precision, recall, F1-score, g-mean, balanced accuracy, selectivity, and error rate

**Fig. 5.** Architecture of LSTM method.

to determine the optimal spam detection approach. Across the experiments on all three datasets spambase, spamassassin, and lingspam the LSTM model consistently achieved the best overall results based on the majority of metrics. LSTM obtained higher accuracy than the other models on each dataset. The key reason LSTM outperformed the alternative methods was its specialized architecture, designed for sequential data. LSTM's memory cell captured long-term dependencies and contextual information in sequences, allowing it to effectively learn the patterns and relationships between words or tokens in email data sequences. As email content consisted of long sequences of text, LSTM's ability to retain and process contextual information over many timesteps made it particularly well-suited for spam detection. The comparative analysis repeatedly demonstrated LSTM's strengths across all datasets.

3.4. Internal architecture of long short-term memory network

The LSTM cells are constructed with a substantial memory cell block, and the two states, namely the hidden state and the cell state, facilitate the flow of data by incorporating slight adjustments via a sequence of multiplication and addition operations. Fig. 5 depicts the architectural structure of an LSTM unit. The forget gate is responsible for eliminating irrelevant and insignificant information from the network by employing channel multiplication. A logistic function incorporates the input at a given time phase (x_t) and the hidden state of the preceding cell (h_{t-1}) to determine which information should be discarded and which information should be retained. Upon reaching the forget state, the cell states undergo the following conclusion [23,24]. The architecture of LSTM is given Fig. 5. The component of the LSTM is given by Eq. (1).

The cell state, C_t , is updated as follows:

$$C_t = f_t \cdot C_{t-1} \quad (1)$$

where f_t and C_{t-1} indicate the forget and previous cell states, respectively. The forget state is defined by the following equation:

$$f_t = \frac{1}{1 + e^{-(W_f \cdot h_{t-1} \cdot x_t + b_f)}} \quad (2)$$

where the weight matrix and bias vector are represented as W_f and b_f , correspondingly. The output vector of the sigmoid function is subjected to a multiplication operation with the cell state, followed by

Table 7
Comparison results of all methods for lingspam dataset.

Methods	Accuracy	Precision	Recall	F1-score	G-mean	Balanced accuracy	Selectivity	Error-rate
ANN	91.64	92.58	82.95	87.5	89.24	89.47	96.00	8.36
LSTM	92.07	89.94	87.60	88.76	90.78	90.84	94.08	7.93
GRU	91.86	91.45	87.82	89.60	91.29	91.36	94.90	8.14
Bi-LSTM	89.04	88.09	86.3	87.19	89.63	89.69	93.08	10.96

Table 8
The summary results obtained by DL methods on the lingspam dataset.

Methods	Best accuracy	Average accuracy	Worst accuracy	Computational times (s)	SD
ANN	91.64	91.19	90.73	1303	0.32
LSTM	92.07	91.88	91.49	1263	0.21
GRU	91.86	91.17	90.26	1297	0.69
Bi-LSTM	89.04	88.61	88.29	1447	0.42

Table 9
Statistical test result of fitness value for spambase dataset.

Methods	t-test
ANN vs. LSTM	0.001
ANN vs. GRU	0.010
ANN vs. Bi-LSTM	0.001
LSTM vs. GRU	0.001
LSTM vs. Bi-LSTM	0.001
GRU vs. Bi-LSTM	0.046

Table 10
Statistical test result of fitness value for spamassassin dataset.

Methods	t-test
ANN vs. LSTM	0.001
ANN vs. GRU	0.019
ANN vs. Bi-LSTM	0.001
LSTM vs. GRU	0.022
LSTM vs. Bi-LSTM	0.001
GRU vs. Bi-LSTM	0.001

Table 11
Statistical test result of fitness value for lingspam dataset.

Methods	t-test
ANN vs. LSTM	0.003
ANN vs. GRU	0.036
ANN vs. Bi-LSTM	0.001
LSTM vs. GRU	0.001
LSTM vs. Bi-LSTM	0.001
GRU vs. Bi-LSTM	0.002

an addition operation with an input gate. The extra sigmoid function is a filtering mechanism that determines which values to incorporate into the cell state and generates a vector using the hyperbolic tangent function that encompasses all potential values. The result of the sigmoid layer and the candidate values for the hyperbolic tangent function are subsequently combined with the cell state. As a result, only essential sequences that are necessary for the cell state are eligible for inclusion. The input state is terminated, resulting in the cell state remaining unchanged, as shown in the following equation:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot C'_t \quad (3)$$

where i_t is the input state, and C'_t carries the candidate values. The two functions are described in the following equations:

$$i_t = \frac{1}{1 + e^{-(W_i \cdot h_{t-1} \cdot x_t + b_i)}} \quad (4)$$

$$C_t = \frac{1 - e^{-2(W_c \cdot h_{t-1} \cdot x_t + b_c)}}{1 + e^{-2(W_c \cdot h_{t-1} \cdot x_t + b_c)}} \quad (5)$$

where the bias vectors b_i and b_c are associated with the input cell and intermediate cell state, respectively. Similarly, the weight matrices W_i

and W_c are linked to the input cell and intermediate cell state. A subsequent filter is employed to determine the specific components of the cell state that should be preserved. Once the hyperbolic tangent function has determined the cell state, the next step involves calculating the sigmoid function and vector. The output of the tanh vector result and the regulating filter are sent into the hidden state as input. This product selectively retains only the most essential sequences. Ultimately, this procedure yields the revised cell state, as shown in Eq. (6):

$$C_t = o_t \cdot h_t \quad (6)$$

This establishes h_t as the hyperbolic tangent vector, and o_t as the regulating filter. The functions for each are given by the following equations:

$$o_t = \frac{1}{1 + e^{-(W_o \cdot [h_{t-1}, x_t] + b_o)}} \quad (7)$$

$$h_t = o_t \cdot \frac{1 - e^{-2C_t}}{1 + e^{-2C_t}} \quad (8)$$

Additionally, W_c and b_o are the weight matrix and bias for the output gate. The researchers suggested modifying the LSTM's peephole connections, which link the cell to the multiplicative gates. This peephole connection was increased to better distinguish between spike sequences. Consequently, the forget, input, and output gates were defined as follows:

$$f_t = \frac{1}{1 + e^{-(W_f \cdot h_{t-1} \cdot x_t + b_f)}} \quad (9)$$

$$i_t = \frac{1}{1 + e^{-(W_i \cdot h_{t-1} \cdot x_t + b_i)}} \quad (10)$$

$$o_t = \frac{1}{1 + e^{-(W_o \cdot [h_{t-1}, x_t] + b_o)}} \quad (11)$$

In an independent study, [25] integrated forget and input gates in their inquiry. Within this particular paradigm, individuals possess the agency to determine which information to disregard and which new material to incorporate into their existing knowledge base. The blended model in question incorporates a forget gate mechanism that selectively discards input sequences, allowing for the injection of fresh values into the state while simultaneously forgetting previously established sequences. Consequently, this model produces the following modified cell state:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot C'_t \quad (12)$$

The following sub-section discusses the property of fuzziness and explains the fuzziness-based divide-and-conquer strategy to better understand the internal calculations.

3.5. Fuzziness-based divide and conquer strategy

Fuzziness refers to the quality of being undefined and lacking clear boundaries. Fuzzy logic is a style of reasoning that attempts to mirror human thinking. The fuzzy set theory handles fuzzy situations by determining the extent to which something belongs to a particular class. There are different techniques to quantify fuzziness. According to [26], fuzziness has three main properties as given in Table 12.

The measure of fuzziness that can be obtained from various fuzzy set applications by applying the following axioms to the function $F \rightarrow [0, 1]^X$:

Table 12
Properties of fuzziness.

Feature 1	Lowest degree of fuzziness. A fuzzy set's all components are either one or zero.
Feature 2	Highest degree of fuzziness. A fuzzy set's all components have an equal value.
Feature 3	Fuzziness between highest and lowest degree. A fuzzy set's all components have values that are neither one nor zero.

- **Axiom 1:** $F(\mu) = 0$ if and only if μ represents a crisp set.
- **Axiom 2:** $F(\mu)$ reaches its maximum value if and only if $\mu(x) = 0.5 \forall x \in X$.
- **Axiom 3:** If $\mu \leq \sigma$, then $F(\mu) \geq F(\sigma)$.
- **Axiom 4:** $F(\mu) = F(\mu')$, where $\mu'(x) = 1 - \mu(x)$ for $\forall x \in X$.
- **Axiom 5:** $F(\mu \cup \sigma) + F(\mu \cap \sigma) = F(\mu) + F(\sigma)$.

The fuzziness of a fuzzy set defined by Eq. (13) attains its maximum when the membership degree of every element is $\mu_i = 0.5$, for every $i = 1, 2, \dots, n$, and attains its minimum when every element either fully belongs to the fuzzy set or does not belong at all, i.e., for every $\mu_i = 0$ or $\mu_i = 1$, for $i(1 \leq i \leq n)$.

Wang et al. proposed a divide-and-conquer methodology to address the complexities associated with fuzziness in the domain of classification. The testing data was partitioned into three distinct groups, wherein each sample was categorized according to its level of fuzziness: low, medium, or high fuzzy value. The inclusion of low and high fuzzy samples from the testing set in the training set serves to improve accuracy [27].

In order to enhance the classification process, the researchers computed the degree of fuzziness pertaining to the prediction values obtained from the employed methodology. The output of the classifier is a fuzzy vector represented as $V = \{\mu_1, \mu_2, \dots, \mu_n\}$. The fuzziness of all samples was determined by utilizing the prediction values derived from the test data and subsequently categorized into three distinct groups: high, low, and medium. Eq. (13) was utilized to quantify the level of fuzziness inherent in a fuzzy event throughout the assessment process:

$$F(V) = -\frac{1}{n} \sum_{i=1}^n \mu_i \log(\mu_i) + (1 - \mu_i) \log(1 - \mu_i) \quad (13)$$

In Eq. (13), the variable n is the total number of classes, whereas μ_i represents the predicted values obtained from the test set for each individual sample using the model. A comprehensive range of fuzziness values, ranging from 0 to 1, was created for all outputs of the model. A threshold was established to distinguish between low fuzzy and high fuzzy samples. The samples with low fuzziness and high fuzziness were transferred from the test set to the training set in order to undergo retraining.

Eq. (13) can be formulated when $n = 2$ and V is normalized such that $\mu_1 + \mu_2 = 1$, the following expressions can be derived:

$$F_1(V) = 1 - \frac{\mu_2}{1 - (1 - \mu_1)^2} \quad (14)$$

$$F_2(V) = \frac{\mu_1}{1 - \mu_1}, \quad 0 \leq \mu_1 \leq 0.5 \quad (15a)$$

$$F_2(V) = \frac{1 - \mu_1}{\mu_1}, \quad 0.5 \leq \mu_1 \leq 1 \quad (15b)$$

To extend this concept, fuzziness is associated with the output of a classifier. Various studies in the literature indicate that many classifiers produce outputs analogous to fuzzy vectors, where each component of the vector represents the membership degree of a given test sample to a specific class C [28]. Given a training set $\{X_i\}_{i=1}^N$, a fuzzy partition assigns a membership degree to each sample concerning the C classes. This partition can be expressed using a membership matrix:

$$U = (U_{ij})_{C \times N}$$

The elements of the membership matrix U must satisfy the following conditions:

$$\sum_{i=1}^C \mu_{ij} = 1, \quad 0 < \sum_{j=1}^N \mu_{ij} < N, \quad \mu_{ij} \in [0, 1]$$

where $\mu_{ij} = \mu_i(x_j)$ denotes the membership of the j th sample x_j in the i th class. Once the classifier completes its training process, the membership matrix U is obtained. During testing, the classifier outputs a fuzzy vector for each j th sample. Based on Eq. (13), the fuzziness of each output vector can be computed using the following equation:

$$F(\mu_j) = -\frac{1}{C} \sum_{i=1}^C (\mu_{ij} \log(\mu_{ij}) + (1 - \mu_{ij}) \log(1 - \mu_{ij})) \quad (16)$$

A unique approach utilizing the divide-and-conquer technique has been devised to address the issue of ambiguity in fuzziness. In this approach, the fuzziness of test samples T_s is categorized into three distinct categories (High, Low, and Medium) based on their respective fuzziness values, as determined by the adopted methodology. Subsequently, samples exhibiting a reduced level of fuzziness were incorporated into the first training set and employed for retraining.

According to Wang et al. the utilization of retraining techniques involving data with low fuzziness can effectively decrease the likelihood of misclassifying such samples. This, in turn, leads to enhanced classification performance and facilitates more precise categorization. The working method is depicted in algorithm 2. Denote the training set as T_r , the testing set as T_s , and the classifier as C [27].

Algorithm 2 Fuzziness Based Divide and Conquer Strategy

- 1: **Input:** Classifier C , Training set T_r , Testing set T_s
- 2: **Output:** New training set T'_r , New accuracy values for T'_r and T_s
- 3: **Step 1:** Distinguish the appropriate fuzzy membership matrix from classifier C on T_s
- 4: **Step 2:** Calculate the accuracy of T_r (TrAcc), T_s (TsAcc), and fuzziness for each output matrix
- 5: **Step 3:** Separate T_s performance into three fuzziness levels:
Low fuzziness: F_{low}
Medium fuzziness: F_{mid}
High fuzziness: F_{high}
- 6: **Step 4:** Create a new training set T_r for the classification with the highest accuracy
- 7: **Step 5:** Obtain new T'_r accuracy (Tr'Acc) and T_s accuracy (TsAcc)
- 8: **Step 6:** Compare exactitude from steps 3 and 5

3.6. Proposed method

In previous research, many efforts focused on improving DL method performance by exploring hybridization techniques and adding layers. These strategies aimed to enhance the method's capabilities and address specific limitations inherent to certain algorithms. One significant advancement in this area was the integration of a fuzziness-based divide-and-conquer strategy with the LSTM network, which culminated in the development of a hybrid method known as the Fuzzy LSTM (FLSTM) method. This approach combined the advantages of both fuzziness and the sequential learning capabilities of LSTM, providing a comprehensive solution to some of the challenges faced by the traditional LSTM method. The fuzziness aspect introduced an interpretative layer to the model, which was particularly useful for handling ambiguous or borderline cases in the data. In a standard LSTM, the method might struggle with uncertain or overlapping data points, potentially leading to classification errors. However, by incorporating fuzziness, the method could effectively separate data into low and high-fuzzy samples, allowing it to make more informed decisions. This was especially beneficial for email spam detection, where certain emails fell

into a gray area with characteristics of both spam and non-spam. The fuzziness component allowed the method to classify these challenging samples more accurately by assigning them degrees of belonging to the spam or non-spam categories rather than forcing a rigid binary classification. LSTM's role in this hybrid approach remained crucial, as it retained its ability to process sequential data effectively. LSTM network was well-known for their strength in handling time-dependent or sequential patterns, which was essential for analyzing email content that often included lengthy, structured text sequences. The memory cells within LSTM allowed the model to retain information from earlier parts of the sequence and apply it to later parts, enabling the model to understand contextual relationships within emails. By combining this sequential learning with the fuzziness component, the Fuzzy LSTM method capitalized on LSTM's strength while overcoming its limitations in ambiguous cases. This study introduced this fuzziness-based LSTM method specifically for email spam detection known as (FLSTM-ESD). The method performance was enhanced by detecting spam emails more effectively and by retraining it with low and high fuzzy samples. This retraining process allowed the method to improve iteratively, as the low fuzzy samples represented data points where the classification was confident, while the high fuzzy samples represented more ambiguous cases. By leveraging both types of samples, the classifier continuously refined its ability to distinguish spam from legitimate emails, learning from both clear-cut examples and nuanced cases. This ongoing adaptation made the method highly robust and well-suited to the ever-changing nature of spam content, which included diverse language patterns, topics, and tactics. Moreover, the fuzziness-based LSTM method provided a more human-interpretable approach to spam detection. Fuzziness allowed the classifier to produce outputs with varying degrees of confidence, offering insight into why certain emails were classified as spam or non-spam. This was valuable for users and administrators who might want to review borderline cases or set thresholds for automatic actions based on the classifier's confidence levels. Thus, the hybrid Fuzzy LSTM method not only boosted detection accuracy but also enhanced transparency, providing a more flexible and adaptable tool for spam detection. By combining fuzziness with LSTM, this research offered an effective approach to email spam detection. The fuzziness component aided in handling ambiguous cases, while the LSTM's architecture captured the sequential dependencies within email data. The classifier was further strengthened by iteratively retraining on low and high fuzzy samples, which equipped it with the ability to adapt over time. This ensured that the model remained effective even as new spam patterns emerged, making it a highly valuable tool in combating spam and maintaining the integrity of email systems. The operational procedures of the proposed methodology are illustrated in algorithm 3. The working process of the proposed method was further elucidated in Fig. 6.

3.7. Method optimization

Method optimization aimed to maximize efficiency while minimizing complexity. For FLSTM-ESD, optimizing the method was of crucial significance, with an emphasis on critical factors such as batch size, dropout, and the number of epochs. The number of epochs impacted method performance and training time, with too few potentially leading to underfitting and too many to overfitting. Batch size affected both method performance and computational efficiency, while dropout served as a regularization technique to prevent overfitting. The optimization of these parameters often required systematic experimentation to balance method complexity, training efficiency, and overall performance. The hyper-parameter tuning for email spam detection in FLSTM-ESD is presented in Table 13. The optimal parameter values were determined based on their effectiveness during training on the spambase, spamassassin, and lingspam datasets. A batch size of 16 was selected as it provided a balance between update frequency and

Algorithm 3 Pseudocode of FLSTM-ESD method

```

1: Input: Email spam datasets
2: Output: Detect email spam or ham and improve accuracy for email classification
3: Step 1: UnderstandDataset(dataset)
4: Step 2: Preprocess the data
5:   handleMissingValues, stopwordsRemoval,
   tokenization, word2vec
6: Step 3: Split the dataset into training set  $\rightarrow T_r$  and testing set  $\rightarrow T_s$ 

7: Step 4: Create a classifier  $C$  for email spam
8: Step 5: Train the classifier  $C$  on  $T_r$  using a training method
9: Step 6: Obtain training accuracy  $\rightarrow \text{TrAcc}$ 
10: Step 7: Build Fuzzy membership vector using divide and conquer strategy
11: for each sample in  $T_s$  do
12:   Acquire the Fuzzy Vector list  $F_i = f(x_1), f(x_2), \dots, f(x_n)$ 
   employing classifier  $C$ 
13: end for
14: for each sample in  $T_s$  do
15:   Calculate Fuzzy vector  $F$  by Equation (5.13)
16: end for
17: Determine fuzzy values in  $T_s$  using the equation of  $F$ 
18: Identify fuzzy samples from  $T_s$ 
19: Build new training set  $T'_r$ , where  $T'_r = T_r \cup (T_s(\text{low}) + T_s(\text{high}))$  by
   enhancing  $T_s(\text{low})$  and  $T_s(\text{high})$ 
20: Retrain classifier on  $T'_r$  using the training method
21: Acquire new training and testing accuracy with  $T'_r$ 
22: Step 8: Evaluate the method

```

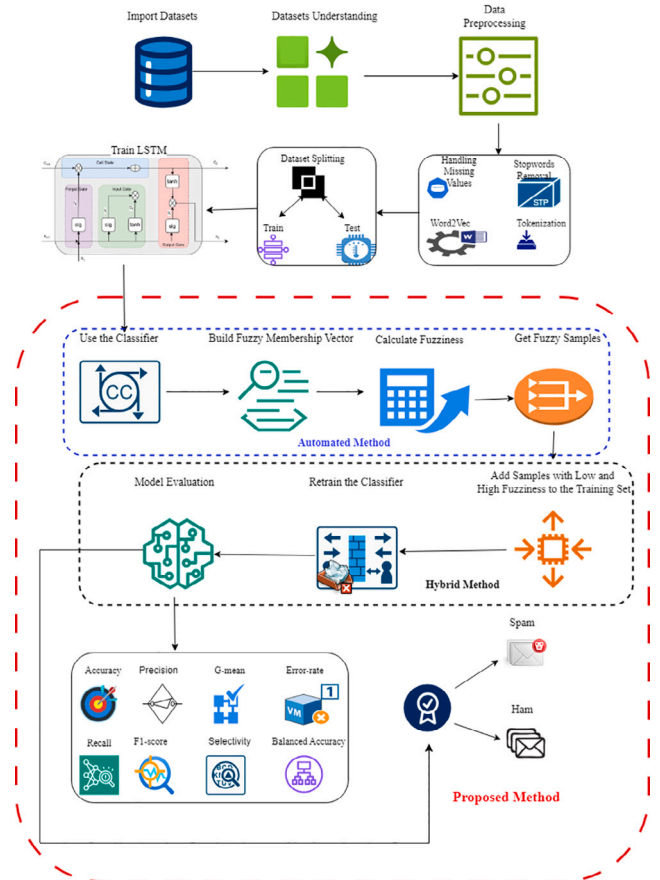


Fig. 6. Flowchart of FLSTM-ESD method.

Table 13
Hyper-parameter settings for email spam detection.

Datasets	Parameter	Utilized values	Optimal value
Spambase	Batch size	8, 16, 32	16
	Dropout	0.1, 0.2, 0.3, 0.4	0.2
	Number of epochs	30, 40, 50, 60	50
	Optimizer	Adam, RMSprop, SGD	RMSprop
Spamassassin and lingspam	Batch size	8, 16, 32	16
	Dropout	0.1, 0.2, 0.3, 0.4	0.2
	Number of epochs	5, 10, 15, 20	15
	Optimizer	Adam, RMSprop, SGD	Adam

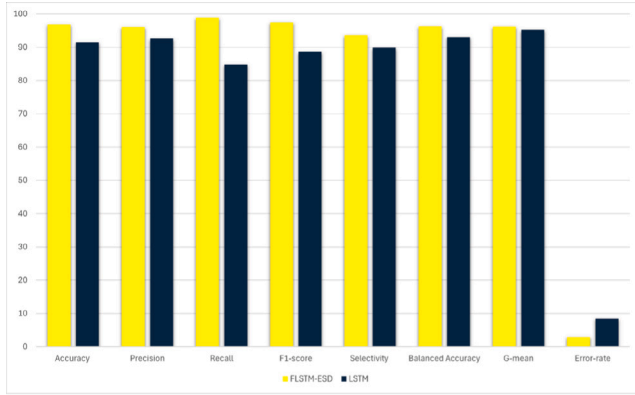


Fig. 7. Comparison of metrics measurement for spamassassin dataset.

memory efficiency, enabling the model to make effective weight adjustments without overloading resources. The dropout rate of 0.2 offered just enough regularization to prevent overfitting while preserving the method capacity to learn key features. Regarding the number of epochs, 50 was ideal for spambase due to its higher complexity and the need for extended training to capture nuanced patterns. For the spamassassin and lingspam datasets, 15 epochs sufficed, allowing the method to converge efficiently while avoiding overfitting. The choice of optimizer varied by dataset; RMSprop was optimal for spambase because of its adaptive learning rate, which managed the dataset's non-stationary nature by responding effectively to changes in data distribution. Meanwhile, Adam was more suitable for spamassassin and lingspam, as it combined adaptive learning with faster convergence, capturing the dataset patterns effectively with fewer epochs. This combination of values enabled the models to achieve consistent, high accuracy across datasets, underscoring their generalizability in email spam detection tasks.

3.8. Results analysis

Evaluating the effectiveness of the proposed method, three datasets—Lingspam, Spamassassin, and Spambase were employed. As discussed in the comparative analysis, LSTM consistently achieved the highest accuracy across all datasets. Following this, FLSTM and LSTM were compared for each dataset, and their performance was further contrasted with existing methods in the literature. Convergence graphs were used to visually depict the performance of the proposed method, providing a clear representation of its efficacy across the various datasets.

Fig. 7 presented a bar chart for the spamassassin dataset, which also compared the performance of FLSTM-ESD and LSTM across the same metrics. Each pair of columns (yellow for FLSTM-ESD and dark blue for LSTM) represented a specific metric, allowing for a direct comparison. This chart offered a clearer, more precise visualization of the difference in performance between the two methods.

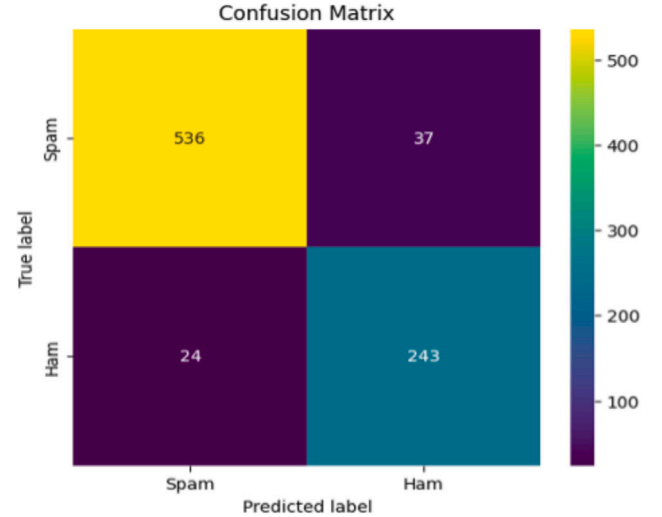


Fig. 8. Confusion matrix for spamassassin dataset using FLSTM-ESD.

FLSTM-ESD outperformed LSTM across all metrics on the spamassassin dataset, as demonstrated in Table 14. FLSTM-ESD achieved higher accuracy (96.84% compared to 91.53%), precision (96.08% compared to 92.6%), and recall (98.89% compared to 84.85%), while also significantly reducing the error rate (3.16% compared to 8.47%). This indicated that FLSTM-ESD was the more effective method for email spam detection.

Table 15 compared the performance of the LSTM and FLSTM-ESD methods on the spamassassin dataset. FLSTM-ESD achieved the highest best accuracy of 96.84%, while LSTM reached 91.53%. The average accuracy for FLSTM-ESD was also higher at 96.68%, compared to 91.28% for LSTM. In terms of computational time, LSTM was faster, taking 1207 s, whereas FLSTM-ESD required 1520 s. However, FLSTM-ESD demonstrated better consistency with a lower SD of 0.12, compared to LSTM's 0.18. Fig. 8 illustrates the strong performance of the FLSTM-ESD method in spam detection.

Table 16 provided a comparative analysis of the accuracy achieved by various methods when applied to the spamassassin dataset, in relation to results reported by previous studies. The table listed several approaches, including NB and LSTM, which achieved an accuracy of 94.05%. Another method, the Multi-Layer Batch Classifier Aggregator with Subspace Randomization (MLBCA-SR), yielded a significantly lower accuracy of 77.51%. The Genetic Algorithm (GA) and Random Weight Networks (RWN) methods demonstrated a respectable accuracy of 93.30%. Additionally, the Deep Belief Network (DBn) combined with a Recurrent Deep Neural Network (RDNN) achieved an accuracy of 96.1%. However, the FLSTM-ESD method introduced in this study outperformed all these approaches, with the highest accuracy of 96.84%. The FLSTM-ESD method offered a superior approach to spam detection on the spamassassin dataset when compared to these previous methods.

Fig. 9 presents a bar chart to offer a more granular, numerical comparison of the performance of FLSTM-ESD and LSTM across all

Table 14

Comparison results of all metrics for spamassassin dataset.

Methods	Accuracy	Precision	Recall	F1-score	G-mean	Balanced accuracy	Selectivity	Error-rate
LSTM	91.53	92.6	84.85	88.55	89.86	93.02	95.16	8.47
FLSTM-ESD	96.84	96.08	98.89	97.47	93.56	96.22	96.19	2.91

Table 15

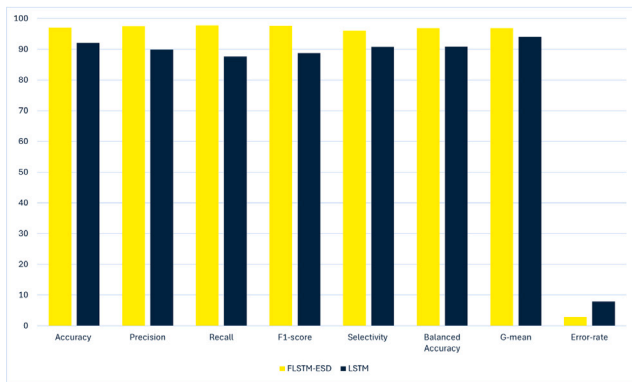
The summary results of the spamassassin dataset.

Methods	Best accuracy	Average accuracy	Worst accuracy	Computational times (s)	SD
LSTM	91.53	91.28	90.96	1207	0.18
FLSTM-ESD	96.84	96.68	96.49	1520	0.12

Table 16

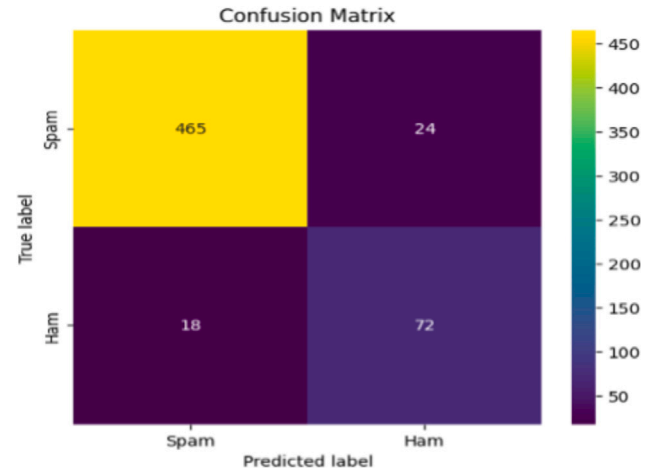
Comparison of the results achieved when applied to the spamassassin dataset with previous studies.

Method	Accuracy (%)
NB, LSTM [29]	94.05
MLBCA-SR [30]	77.51
GA, RWN [31]	93.30
DBN-RDNN-ReL [32]	96.1
FLSTM-ESD	96.84

**Fig. 9.** Comparison of metrics measurement for lingspam dataset.

metrics. Each column represented a performance metric, and the differences between the two methods were clearly visible. FLSTM-ESD, indicated by yellow columns, consistently achieved higher values than LSTM (dark blue columns), which experimentally confirmed its superior performance.

Table 17 presents a detailed comparison of the performance of LSTM and FLSTM-ESD across multiple metrics when applied to the lingspam dataset. The FLSTM-ESD method demonstrated significant improvements over the LSTM method in all metrics. Specifically, FLSTM-ESD achieved an accuracy of 97.09%, outperforming LSTM, which achieved 92.07%. The Precision metric for FLSTM-ESD was 97.5%, compared to LSTM's 89.94%, indicating that FLSTM-ESD was more precise in correctly identifying spam emails. Similarly, FLSTM-ESD showed superior recall, with a score of 97.78% versus LSTM's 87.6%, suggesting that FLSTM-ESD was more effective in correctly capturing true positive spam messages. This higher recall contributed to a better F1-score of 97.64% for FLSTM-ESD, as opposed to LSTM's 88.76%, showing that FLSTM-ESD struck a better balance between precision and recall. The g-mean metric for FLSTM-ESD was also higher, at 96%, compared to LSTM's 90.78%, demonstrating that FLSTM-ESD performed consistently well across both spam and ham classes. Furthermore, FLSTM-ESD achieved a balanced accuracy of 96.89%, surpassing LSTM's 90.84%, and maintained a higher selectivity (96.89% compared to LSTM's 94.08%). Notably, FLSTM-ESD significantly reduced the

**Fig. 10.** Confusion matrix for lingspam dataset using FLSTM-ESD.

error rate to 2.91%, compared to LSTM's much higher error rate of 7.93%. FLSTM-ESD was the superior method for detecting email spam.

Table 18 compared the performance of LSTM and FLSTM-ESD methods on the lingspam dataset. FLSTM-ESD outperformed LSTM across all key metrics, achieving a best accuracy of 97.09%, while LSTM reached 92.07%. The average accuracy for FLSTM-ESD was 96.89%, higher than LSTM's 91.88%. FLSTM-ESD had a lower SD of 0.16 compared to LSTM SD of 0.21, indicating more consistent performance. However, LSTM required less computational time, completing in 1263 s, while FLSTM-ESD took 1493 s, reflecting the trade-off between accuracy and computational efficiency. Fig. 10 demonstrates the high efficacy of the FLSTM-ESD approach in identifying spam.

Table 19 provided a detailed comparison of the accuracy achieved by various ML methods when applied to the lingspam dataset, based on results reported in previous studies. This comparison highlighted the performance of different methods such as GA, LSTM, SVM, k-NN, NB, DT, RF, and BERT. These methods had historically been effective for spam detection, with accuracies ranging from 94% to 96.8%. However, the FLSTM-ESD method introduced in this study surpassed all of these approaches, achieving the highest accuracy at 96.84%. This result underscored the superiority of FLSTM-ESD, demonstrating its ability to more accurately classify spam and ham messages in the lingspam dataset. The improvement in accuracy suggested that the modifications made in the FLSTM-ESD architecture provided a more robust method for handling the complexities of spam detection, potentially due to its enhanced ability to capture both temporal and contextual features within the data.

Fig. 11 illustrates a bar chart to provide a clearer and more quantitative comparison of the same metrics for both FLSTM-ESD and LSTM for spambase. Each metric was represented by pairs of columns, with the yellow column indicating FLSTM-ESD and the dark blue column indicating LSTM. This chart reinforced the findings of the radar chart, showing that FLSTM-ESD consistently outperformed LSTM across most metrics. The most notable improvement was seen in the error rate, where FLSTM-ESD achieved a significantly lower value compared to LSTM.

Table 20 presented a comparison of various performance metrics between the LSTM and FLSTM-ESD methods on the spambase dataset. FLSTM-ESD outperformed LSTM across all key metrics. Specifically,

Table 17
Comparison results of all metrics for lingspam dataset.

Methods	Accuracy	Precision	Recall	F1-score	G-mean	Balanced accuracy	Selectivity	Error-rate
LSTM	92.07	89.94	87.6	88.76	90.78	90.84	94.08	7.93
FLSTM-ESD	97.09	97.5	97.78	97.64	96	96.89	96.89	2.91

Table 18
The summary results of the lingspam dataset.

Methods	Best accuracy	Average accuracy	Worst accuracy	Computational times (s)	SD
LSTM	92.07	91.88	91.49	1263	0.21
FLSTM-ESD	97.09	96.89	96.67	1493	0.16

Table 19
Comparison of the results achieved when applied to the lingspam dataset with previous studies.

Method	Accuracy (%)
GA, RWN [31]	96.80
BERT [33]	95.77
SVM, k-NN, NB, DT, RF, AB [34]	94
LSTM, BERT [35]	96
FLSTM-ESD	97.09

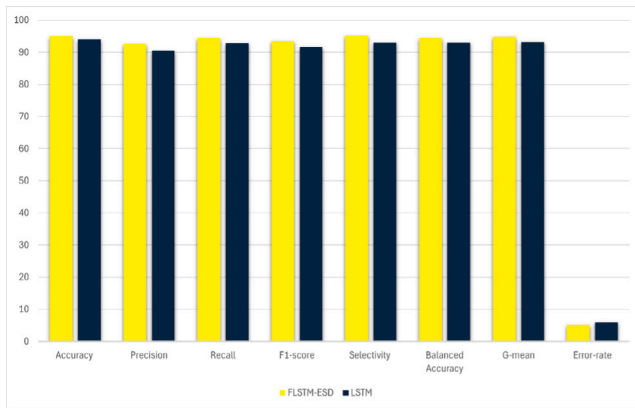


Fig. 11. Comparison of metrics measurement for spambase dataset.

FLSTM-ESD achieved a higher accuracy of 94.9% compared to LSTM's 94.03%. The precision of FLSTM-ESD was also higher at 92.58%, indicating that it was better at correctly identifying spam messages, as compared to LSTM's 90.46%. Additionally, FLSTM-ESD showed improved recall, scoring 94.4%, which suggested that it was more effective in correctly detecting spam, compared to LSTM's recall of 92.86%. FLSTM-ESD also led in the F1-score metric, with a score of 93.48% compared to LSTM's 91.65%, reflecting a better balance between precision and recall. Furthermore, FLSTM-ESD had a higher G-mean of 95.21%, suggesting better overall performance across both spam and non-spam classifications, as compared to LSTM's 93.04%. In terms of balanced accuracy, FLSTM-ESD achieved 94.4%, outperforming LSTM's 93.02%. The selectivity metric also favored FLSTM-ESD with a score of 94.8% compared to LSTM's 93.19%, indicating better discrimination between spam and ham classes. Finally, the error rate was significantly lower for FLSTM-ESD at 5.1%, compared to LSTM's 5.97%.

Table 21 compared the performance of LSTM and FLSTM-ESD on the spambase dataset. FLSTM-ESD achieved a higher best accuracy of 94.90% compared to LSTM's 94.03%. Additionally, FLSTM-ESD had a slightly higher average accuracy (94.56%) than LSTM (93.51%). However, LSTM was faster in terms of computational time, requiring 652 s, while FLSTM-ESD took 673 s. Both methods had similar consistency, with FLSTM-ESD showing an SD of 0.31, and LSTM having a SD of 0.32. FLSTM-ESD required more computational time compared to LSTM

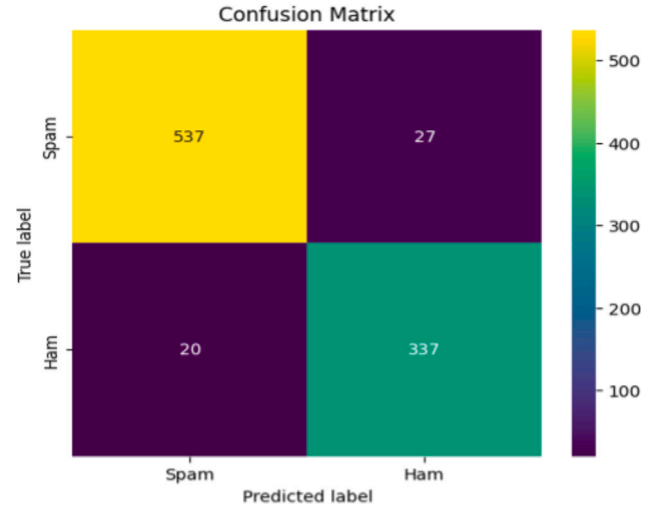


Fig. 12. Confusion matrix for spambase dataset using FLSTM-ESD.

due to the additional complexity introduced by the fuzziness-based mechanism integrated into its architecture. These extra computational tasks contribute to a longer processing time, which, while increasing computational cost, allows FLSTM-ESD to achieve higher accuracy and better performance across various metrics compared to the standard LSTM. Fig. 12 illustrates the strong performance of the FLSTM-ESD method in spam detection.

A comparison of the accuracy achieved by different methods when applied to the spambase dataset, as reported in previous studies, was presented in Table 22. The table showed that the Random Forest (RF) and DNN methods achieved an accuracy of 88.59%, which was significantly lower compared to the other methods. The Support Vector Machine (SVM) and RF methods reached an accuracy of 94%, while the Binary Harris Hawk Optimization (BHBO) method obtained a slightly higher accuracy of 94.30%. Additionally, a Neural Network (NN) method reported an accuracy of 94.55%. The FLSTM-ESD method, developed in this study, achieved the highest accuracy at 94.9%, surpassing all the other methods. This demonstrated that FLSTM-ESD provided the most effective approach for spam detection on the spambase dataset, outperforming both traditional ML methods and recent advanced algorithms. The table highlighted the superiority of FLSTM-ESD in terms of accuracy when compared with the state-of-the-art methods used in previous studies.

A t-test was conducted to determine whether the accuracy of the LSTM method and the FLSTM-ESD method statistically significantly differed. The t-test was based on accuracy results from 30 independent runs per sample, with a significance level (α) set at 0.05 for the paired t-test. If the p -value exceeded α , the null hypothesis H_0 , stating no significant difference between the LSTM and FLSTM-ESD methods, would be accepted. Conversely, if the p -value was less than α , the alternative hypothesis H_1 , stating a significant difference between the methods, would be accepted. For the Spambase dataset, the t-test yielded a p -value of 0.001. Similarly, for the Spambassassin dataset, the p -value was 0.004, and for the Lingspam dataset, it was 0.001. Since all p -values were below the 0.05 threshold, the null hypothesis H_0 was rejected, and the alternative hypothesis H_1 was accepted, indicating significant differences between the methods. These results

Table 20
Comparison results of all metrics for spambase dataset.

Methods	Accuracy	Precision	Recall	F1-score	G-mean	Balanced accuracy	Selectivity	Error-rate
LSTM	94.03	90.46	92.86	91.65	93.04	93.02	93.19	5.97
FLSTM-ESD	94.9	92.58	94.4	93.48	95.21	94.4	94.8	5.1

Table 21
The summary results of the spambase dataset.

Methods	Best accuracy	Average accuracy	Worst accuracy	Computational times (s)	SD
LSTM	94.03	93.51	93.19	652	0.32
FLSTM-ESD	94.90	94.56	94.05	673	0.31

Table 22
Comparison of the results achieved when applied to the spambase dataset with previous studies.

Method	Accuracy (%)
RF, DNN [36]	88.59
SVM, RF [37]	94
BHHO [38]	94.30
NN [39]	94.55
FLSTM-ESD	94.9

confirm that the accuracy of FLSTM-ESD is significantly higher than that of LSTM. FLSTM-ESD achieved the highest accuracy across all three datasets for email spam detection due to its advanced architecture and effective methodology. The method combines the strengths of the LSTM network, which excels at capturing long-range dependencies and contextual information in sequential data, with enhanced feature extraction techniques. This hybrid structure allows FLSTM-ESD to better represent the intricate patterns present in email text, leading to more accurate classification decisions. Additionally, the use of attention mechanisms enables the model to focus on the most relevant parts of the data, improving its ability to identify critical words or phrases that distinguish spam from ham emails. This targeted focus enhances both precision and recall while minimizing errors, which is crucial in real-world spam detection scenarios. Another key factor in FLSTM-ESD's success is its ability to handle class imbalances typically found in spam datasets, where legitimate emails (ham) vastly outnumber spam emails. The method's balanced learning capabilities are reflected in its high G-mean and balanced accuracy scores, showing that it can effectively manage predictions across both classes. Furthermore, FLSTM-ESD demonstrates strong generalization abilities, performing consistently well across different datasets (Spamassassin, Lingspam, and Spambase), indicating its adaptability to various types of email data.

4. Discussion

FLSTM-ESD achieved the highest accuracy across all datasets in our experiments 97.09% for the Lingspam dataset, 96.84% for the Spamassassin dataset, and 94.9% for the Spambase dataset. This outstanding performance was attributed to its innovative combination of LSTM network with fuzzy membership vectors, which significantly enhanced the system's capacity to manage uncertainty and ambiguity within the data. LSTM networks were known for their effectiveness in handling sequential data, such as email text, because they were capable of capturing both short-term and long-term dependencies between words. This was particularly valuable for email spam detection, where context played a critical role in distinguishing between spam and legitimate emails (ham). By retaining and processing long sequences of words and phrases, LSTM models were well-suited to understanding the inherent structure and context within emails, allowing them to detect patterns that simpler models might have overlooked. However, while LSTM excelled in this domain, it still faced challenges when dealing with highly

uncertain or ambiguous data. This was where the integration of fuzzy membership vectors in FLSTM-ESD offered a substantial improvement. Fuzzy logic, unlike traditional binary decision-making models, allowed for varying degrees of belonging, or membership, to multiple classes. In the context of email spam detection, this meant that an email could be assigned partial degrees of belonging to both the spam and ham categories. This was especially useful in cases where emails exhibited characteristics of both classes—such as newsletters or promotional emails that might resemble spam but contained legitimate content. Traditional methods often struggled with such borderline cases, but the fuzzy membership approach provided a more nuanced way of making classification decisions. By allowing for softer decisions rather than forcing a binary classification, FLSTM-ESD reduced the risk of misclassifications, which was one of the main factors contributing to its higher accuracy. The benefit of incorporating fuzzy membership vectors was particularly evident in the ability of FLSTM-ESD to handle complex and noisy data. Spam emails varied widely in format, structure, and language—ranging from blatantly promotional content to more subtle forms that closely resembled legitimate communication. This variety introduced significant uncertainty into the classification process, and rigid, binary classification methods were often ill-equipped to deal with this. Fuzzy membership vectors enabled the model to better navigate this uncertainty by assigning a range of values to an email's classification, reflecting its mixed characteristics. For example, an email might not be clearly spam or ham, but could exhibit elements of both. Fuzzy logic allowed the model to account for this ambiguity and make more informed, accurate decisions, rather than committing to one class or the other too early. Furthermore, the fuzzy membership vectors enhanced LSTM's ability to generalize across diverse datasets. The flexibility provided by fuzzy logic allowed the model to adapt more effectively to the varying characteristics of different datasets, which explained why FLSTM-ESD demonstrated strong performance across all three datasets tested in the study. By improving the model's ability to deal with varied, ambiguous, and noisy data, the integration of fuzzy membership vectors strengthened the decision-making process at its core. It enabled FLSTM-ESD to consistently outperform traditional LSTM networks, as evidenced by the method's superior accuracy. In addition to improving accuracy, the use of fuzzy membership vectors also reduced classification errors, particularly in scenarios where the distinction between spam and legitimate email was unclear. This reduction in error rates was critical for real-world email spam detection systems, where false positives (legitimate emails wrongly classified as spam) and false negatives (spam emails misclassified as ham) both caused significant issues. By incorporating fuzzy logic, FLSTM-ESD struck a better balance between precision (correctly identifying spam) and recall (ensuring spam was not missed), leading to improved overall performance. This sophisticated combination of sequence modeling through LSTM and the nuanced decision-making enabled by fuzzy membership vectors made FLSTM-ESD uniquely capable of handling the complexities of email spam detection. As demonstrated by its performance across the three datasets, FLSTM-ESD not only achieved higher accuracy but also demonstrated superior consistency and generalization compared to traditional methods, offering a robust solution for spam detection. The method's ability to adapt to different types of data while maintaining high accuracy made it a powerful tool for real-world applications, where uncertainty and ambiguity in spam classification were common challenges. The integration of fuzzy membership vectors with LSTM represented a significant advancement in the field of spam detection. By leveraging both the sequence modeling capabilities of the LSTM network and the flexibility of fuzzy logic, FLSTM-ESD provided a more

robust, adaptable, and accurate approach to email classification. This combination reduced errors, improved generalization across datasets, and ultimately led to a more reliable spam detection system. The success of FLSTM-ESD in achieving the highest accuracy on the Lingspam, Spamassassin, and Spambase datasets further highlighted its potential as a superior method for handling the challenges of spam detection in complex, real-world environments.

5. Conclusion and future works

The FLSTM-ESD method successfully combines the strengths of fuzzy logic and LSTM networks to address challenges in traditional spam detection systems. By incorporating fuzzy logic's interpretability and LSTM's sequence modeling capabilities, the method handles uncertainty and captures long-term dependencies in email data. Testing on Spambase, Spamassassin, and Lingspam datasets showed FLSTM-ESD outperforming other methods, with accuracy rates of 94.9%, 96.84%, and 97.09%, respectively. These results highlight FLSTM-ESD's ability to achieve high accuracy, improve generalization, and overcome the overfitting challenges typical in DL models. Additionally, its interpretability, achieved through fuzzy logic, allows for transparent decision-making, a crucial aspect in real-world applications. By combining strong performance with interpretability, FLSTM-ESD represents a significant step forward in spam detection. This study demonstrated that the proposed FLSTM-ESD method achieved results comparable to previous research, but there remain several opportunities for future improvement. One potential direction is integrating advanced natural language processing (NLP) techniques and DL architectures, such as transformers, BERT, or pre-trained embeddings, to enhance the feature extraction capabilities of FLSTM-ESD. These enhancements could improve the detection of more sophisticated spam and generalization across different datasets. Another promising direction is expanding FLSTM-ESD to handle multilingual spam detection, as spam emails increasingly target users in various languages. Modifying the architecture to process multilingual text while maintaining accuracy and efficiency would significantly broaden its applicability. Additionally, integrating Explainable AI (XAI) techniques to improve the transparency of decision-making could enhance the method's interpretability, which is essential for gaining user trust in real-world applications. Finally, optimizing FLSTM-ESD for real-time spam detection in large-scale email systems by improving computational efficiency and scalability could make it suitable for cloud-based deployments, enhancing email security at scale.

CRedit authorship contribution statement

Ekramul Haque Tusher: Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Conceptualization. **Mohd Arfian Ismail:** Writing – review & editing, Project administration, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the Fundamental Research Grant (FRGS) with FRGS/1/2022/ICT02/UMP/02/2 from the Ministry of Higher Education Malaysia under Grant RDU220134. Additional funding was provided by the Postgraduate Research Grant Scheme (PGRS) from University Malaysia Pahang Al-Sultan Abdullah, with vote number PGRS2303110.

Data availability

The data used are public data.

References

- [1] E.H. Tusher, M.A. Ismail, M.A. Rahman, A.H. Alenezi, M. Uddin, Email spam: A comprehensive review of optimize detection methods, challenges, and open research problems, *IEEE Access* (2024).
- [2] P. Dhal, C. Azad, Hybrid momentum accelerated bat algorithm with GWO based optimization approach for spam classification, *Multimedia Tools Appl.* 83 (9) (2024) 26929–26969.
- [3] A.H. Dinnillah, M. Widiarto, A. Wijaya, Robust phishing detection using random forest with isolation forest based outlier filtering, *IJACI: Int. J. Adv. Comput. Informatics* 1 (2) (2025) 99–105.
- [4] T. Muralidharan, N. Nissim, Improving malicious email detection through novel designated deep-learning architectures utilizing entire email, *Neural Netw.* 157 (2023) 257–279.
- [5] N.S. Rahmawati, C.I. Amalia, Lightweight ensemble models for static malware detection: AddressingDeep learning trade-offs with the kaggle PE dataset, *IJACI: Int. J. Adv. Comput. Informatics* 2 (1) (2026) 21–33.
- [6] A. Swaminathan, B. Ramakrishnan, et al., Prediction of cyber-attacks and criminality using machine learning algorithms, in: 2022 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT), IEEE, 2022, pp. 547–552.
- [7] R. Mahaveerakannan, T. Tamilvizhi, S.J. Rayen, O.I. Khalaf, H. Hamam, Information centric networking based cooperative caching framework for 5G communication systems., *Comput. Mater. Contin.* 80 (3) (2024).
- [8] A. Ghosh, A. Senthilrajan, Comparison of machine learning techniques for spam detection, *Multimedia Tools Appl.* 82 (19) (2023) 29227–29254.
- [9] I. AbdulNabi, Q. Yaseen, Spam email detection using deep learning techniques, *Procedia Comput. Sci.* 184 (2021) 853–858.
- [10] T. Vaiyapuri, K. Shankar, S. Rajendran, S. Kumar, V. Gaur, D. Gupta, M. Alharbi, Automated cyberattack detection using optimal ensemble deep learning model, *Trans. Emerg. Telecommun. Technol.* 35 (4) (2024) e4899.
- [11] P.K. Roy, J.P. Singh, S. Banerjee, Deep learning to filter SMS spam, *Future Gener. Comput. Syst.* 102 (2020) 524–533.
- [12] E.H. Tusher, M.A. Ismail, A.F. Mat Raffei, Email spam classification based on deep learning methods: a review, *Iraqi J. Comput. Sci. Math.* 6 (1) (2025) 2.
- [13] G. Alkhodhairi, K. Saleem, Machine learning algorithm for detecting suspicious email messages using natural language processing NLP, *Alex. Eng. J.* 128 (2025) 153–165.
- [14] L. Sankaine, J.G. Ndia, D. Kaburu, An english-swahili email spam detection model for improved accuracy using convolutional neural networks, *Mesopotamian J. CyberSecurity* 5 (2) (2025) 590–605.
- [15] S. Zavrak, S. Yilmaz, Email spam detection using hierarchical attention hybrid deep learning method, *Expert Syst. Appl.* 233 (2023) 120977.
- [16] S. Magdy, Y. Abouelseoud, M. Mikhail, Efficient spam and phishing emails filtering based on deep learning, *Comput. Netw.* 206 (2022) 108826.
- [17] B. Lindemann, B. Maschler, N. Sahlab, M. Weyrich, A survey on anomaly detection for technical systems using lstm networks, *Comput. Ind.* 131 (2021) 103498.
- [18] İ. Atacak, O. Çıtlak, İ.A. Doğru, Application of interval type-2 fuzzy logic and type-1 fuzzy logic-based approaches to social networks for spam detection with combined feature capabilities, *PeerJ Comput. Sci.* 9 (2023) e1316.
- [19] M.J. Patwary, W. Cao, X.-Z. Wang, M.A. Haque, Fuzziness based semi-supervised multimodal learning for patient's activity recognition using RGBDT videos, *Appl. Soft Comput.* 120 (2022) 108655.
- [20] J. Zhou, G. Yuan, C. Gao, X. Wang, J. Dai, W. Pedrycz, Fuzzy clustering guided by spectral rotation and scaling, *IEEE Trans. Cybern.* (2024).
- [21] M. Hu, Y. Guo, R. Wang, X. Wang, Attribute reduction with fuzzy kernel-induced relations, *Inform. Sci.* 669 (2024) 120589.
- [22] K. Hussain, N. Neggaz, W. Zhu, E.H. Houssein, An efficient hybrid sine-cosine harris hawks optimization for low and high-dimensional feature selection, *Expert Syst. Appl.* 176 (2021) 114778.
- [23] S. Wang, C. Ma, H. Gao, D. Deng, C. Fernandez, F. Blaabjerg, Improved hyperparameter Bayesian optimization-bidirectional long short-term memory optimization for high-precision battery state of charge estimation, *Energy* (2025) 136598.
- [24] S. Wang, Y. Fan, S. Jin, P. Takyi-Aninakwa, C. Fernandez, Improved anti-noise adaptive long short-term memory neural network modeling for the robust remaining useful life prediction of lithium-ion batteries, *Reliab. Eng. Syst. Saf.* 230 (2023) 108920.
- [25] Q. Liu, Q. Wang, X. Wang, RVFL-LSTM: A lightweight model with long-short term memory for time series, *Knowl.-Based Syst.* 309 (2025) 112896.
- [26] T. Zhang, X. Wang, Anchorwise fuzziness modeling in convolution-transformer neural network for left atrium image segmentation, *IEEE Trans. Fuzzy Syst.* 32 (2) (2023) 398–408.

- [27] X.-Z. Wang, R.A.R. Ashfaq, A.-M. Fu, Fuzziness based sample categorization for classifier performance improvement, *J. Intell. Fuzzy Systems* 29 (3) (2015) 1185–1196.
- [28] J. Cao, T. Zhou, S. Zhi, S. Lam, G. Ren, Y. Zhang, Y. Wang, Y. Dong, J. Cai, Fuzzy inference system with interpretable fuzzy rules: Advancing explainable artificial intelligence for disease diagnosis—A comprehensive review, *Inform. Sci.* 662 (2024) 120212.
- [29] K.F. Rafat, Q. Xin, A.R. Javed, Z. Jalil, R.Z. Ahmad, Evading obscure communication from spam emails, *Math. Biosci. Eng.* 19 (2) (2022) 1926–1943.
- [30] D. Davino, F. Camastra, A. Ciaramella, A. Staiano, Spam detection by machine learning-based content analysis, in: *Progresses in Artificial Intelligence and Neural Systems*, Springer, 2020, pp. 415–422.
- [31] H. Faris, A.-Z. Ala'M, A.A. Heidari, I. Aljarah, M. Mafarja, M.A. Hassonah, H. Fujita, An intelligent system for spam detection and identification of the most relevant features based on evolutionary random weight networks, *Inf. Fusion* 48 (2019) 67–83.
- [32] A. Barushka, P. Hajek, Spam filtering using integrated distribution-based balancing approach and regularized deep neural networks, *Appl. Intell.* 48 (10) (2018) 3538–3556.
- [33] Q. Tang, X. Li, An investigation of large language models and their vulnerabilities in spam detection, 2025, arXiv preprint [arXiv:2504.09776](https://arxiv.org/abs/2504.09776).
- [34] N. Kumar, S. Sonowal, et al., Email spam detection using machine learning algorithms, in: *2020 Second International Conference on Inventive Research in Computing Applications, ICIRCA, IEEE, 2020*, pp. 108–113.
- [35] S.A. Khan, K. Iqbal, N. Mohammad, R. Akbar, S.S.A. Ali, A.A. Siddiqui, A novel fuzzy-logic-based multi-criteria metric for performance evaluation of spam email detection algorithms, *Appl. Sci.* 12 (14) (2022) 7043.
- [36] S. Srinivasan, V. Ravi, M. Alazab, S. Ketha, A. Al-Zoubi, S. Kotti Padannayil, Spam emails detection based on distributed word embedding with deep learning, in: *Machine Intelligence and Big Data Analytics for Cybersecurity Applications*, Springer, 2020, pp. 161–189.
- [37] O. Taylor, P. Ezekiel, A model to detect spam email using support vector classifier and random forest classifier, *Int. J. Comput. Sci. Math. Theory* 6 (1) (2020) 1–11.
- [38] A.S. Mashaleh, N.F.B. Ibrahim, M.A. Al-Betar, H.M. Mustafa, Q.M. Yaseen, Detecting spam email with machine learning optimized with harris hawks optimizer (HHO) algorithm, *Procedia Comput. Sci.* 201 (2022) 659–664.
- [39] F.E. Ayo, L.A. Ogundele, S. Olakunle, J.B. Awotunde, F.A. Kasali, A hybrid correlation-based deep learning model for email spam classification using fuzzy inference system, *Decis. Anal. J.* 10 (2024) 100390.



Ekramul Haque Tusher holds a B.Sc. in Computer Science from International Islamic University Chittagong (IIUC), where he currently serves as an Adjunct Faculty member. He earned his Master's by Research in Soft Computing and Intelligent Systems from Universiti Malaysia Pahang Al-Sultan Abdullah (UMPASA), Pekan, Pahang, Malaysia. Since 2023, Mr. Ekramul has been a Research Assistant with the Machine Intelligence Research Group (MIRG) at UMPASA, where he actively contributes to cutting-edge investigations in artificial intelligence. His research interests span Machine Learning, Deep Learning, Fuzzy Systems, Large Language Model (LLM), and Explainable AI (XAI) — with a particular focus on building intelligent systems that are both powerful and interpretable. He can be contacted at email: ekramulhaquetusher@gmail.com.



Mohd Arfian Ismail is an Associate Professor at the Faculty of Computing in University Malaysia Pahang Al-Sultan Abdullah, Malaysia. He received B. Sc, M. Sc. and PhD degree in Computer Science from Universiti Teknologi Malaysia (UTM) in 2008, 2011 and 2016, respectively. His current research interests are in the areas of machine learning methods and Fuzzy System. He can be contacted at arfian@umpasa.edu.my.